Tracing Mutation and Side Effects

Lecture 21

CS 1001

Kirk Duran

Today: Follow a Changing Reference Diagram

Lecture 20 gave us the rules. Today we combine them in longer traces.

- Follow aliases as names move between lists.
- Separate a function's mutations from its returned value.
- Trace two parameters that may refer to the same list.
- Find the first unintended mutation and place a copy before it.
- Trace repeated calls using a binding ledger and object labels.

In class

Predict first, draw references, then compare with the reveal. Explain the statement responsible for each change.

One-Minute Toolkit

Statement	What to record
`b = a`	Bind `b` to the object currently named by `a`.
`b = a.copy()`	Create a new list; bind `b` to it.
`a[0] = 7`	Change the existing list's contents.
`a = a + [7]`	Create a new list; move the `a` arrow.
`return a`	Give the caller a reference to the current object.

Key idea

Label the lists we track L1, L2, ... as they appear. Keep a **binding ledger** and a separate record of each list's **contents**.

Our examples use flat lists of integers: copies have independent list state.

Warm-Up: The Alias Changes Halfway Through

Code

```
tray = [3, 6, 9]
saved = tray
tray = tray + [12]
saved.pop(0)
tray[1] = saved[1]
print(tray)
print(saved)
```

In class

Predict both outputs. At which line do `tray` and `saved` stop sharing a list?

- Draw a new box for concatenation.
- Recompute indexes after `pop(0)`.
- Read the right-hand side before replacing an element.

Warm-Up: Keep Bindings and Contents Separate

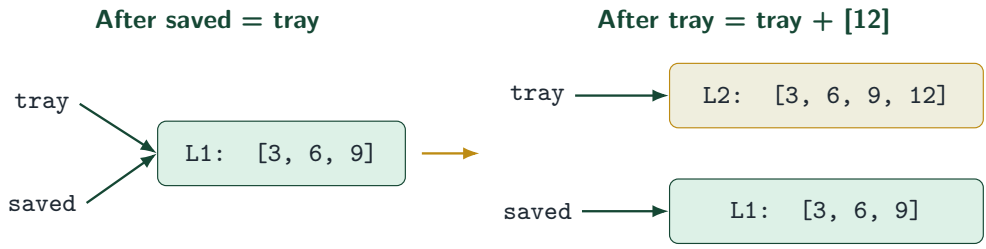
After statement	Bindings	L1	L2
`saved = tray`	`tray, saved -> L1`	`[3, 6, 9]`	—
`tray = tray + [12]`	`tray -> L2; saved -> L1`	`[3, 6, 9]`	`[3, 6, 9, 12]`
`saved.pop(0)`	unchanged	`[6, 9]`	`[3, 6, 9, 12]`
`tray[1] = saved[1]`	unchanged	`[6, 9]`	`[3, 9, 9, 12]`

Output

```
[3, 9, 9, 12]
[6, 9]
```

After the removal, `saved[1]` reads 9 from L1. The assignment writes that value into index 1 of L2.

Picture the Moment a Name Moves



Key idea

Rebinding `tray` leaves `saved` pointing to L1. Later operations must follow the arrows that exist *at that time*.

Part 1: Mutation, Rebinding, and Return

Trace 1: Mutate First, Then Rebind a Parameter

Code

```
def tag(values: list[int]) -> list[int]:
    values.append(4)
    values = values + [5]
    values[0] = 0
    return values

batch = [7, 8]
watch = batch
tagged = tag(batch)
print(batch)
print(tagged)
```

In class

Predict both outputs and `watch is tagged`.
Which mutation reaches the caller's original list? Which reaches only the returned list?

Trace 1: A Function Can Affect Two Different Lists

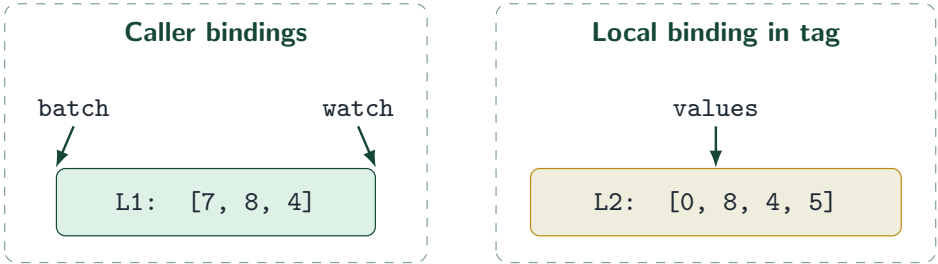
Inside `tag`	Parameter	L1	L2
enter call	`values -> L1`	`[7, 8]`	—
`values.append(4)`	`values -> L1`	`[7, 8, 4]`	—
`values = values + [5]`	`values -> L2`	`[7, 8, 4]`	`[7, 8, 4, 5]`
`values[0] = 0`	`values -> L2`	`[7, 8, 4]`	`[0, 8, 4, 5]`

Output and identity

`batch: [7, 8, 4] tagged: [0, 8, 4, 5]`
`watch is tagged` is `False`.

`batch` and `watch` remain bound to L1 throughout the call.

Trace 1: Just Before the Return



append(4) changed this list

The return will give tagged this reference

Key idea

Moving the local parameter's arrow does not move the caller's arrows. The earlier append has already changed L1.

Quick Check: What If We Ignore the Return?

Use the same `tag` function with a fresh list.

Caller code

```
batch = [7, 8]
tag(batch)
print(batch)
```

In class

Does ignoring the returned value prevent the append? Predict the output before revealing it.

Quick Check: What If We Ignore the Return?

Use the same `tag` function with a fresh list.

Caller code

```
batch = [7, 8]
tag(batch)
print(batch)
```

In class

Does ignoring the returned value prevent the append? Predict the output before revealing it.

Reveal

`[7, 8, 4]` — the append happened during the call. The caller does not save a reference to the returned list.

Part 2: Two Parameters, One Possible List

Trace 2: Start with Different Arguments

Code

```
def transfer(source: list[int], target: list[int]) -> int:
    moved = source.pop(0)
    target.append(moved + 10)
    return moved

left = [2, 5]
right = [8]
number = transfer(left, right)
```

In class

Predict `left`, `right`, and `number`. Record the list contents immediately after each function statement.

Trace 2: Different Arguments, Different Objects

Checkpoint	**L1:** `left/source`	**L2:** `right/target`	`moved`
entry	`[2, 5]`	`[8]`	—
after `pop(0)`	`[5]`	`[8]`	2
after `append`	`[5]`	`[8, 12]`	2

After the call

`left == [5]` `right == [8, 12]` `number == 2`

Key idea

A call can produce a returned value and also mutate its arguments. Track both results.

Trace 3: Pass the Same Argument Twice

The function body is unchanged.

Fresh caller code

```
shared = [2, 5]
number = transfer(shared, shared)
print(shared)
print(number)
```

In class

Draw `shared`, `source`, and `target` at function entry. How many list objects exist? Trace the statements in order.

Trace 3: Both Parameters Observe Each Mutation

Checkpoint	**L1:** `shared/source/target`	`moved`
entry	`[2, 5]`	—
`moved = source.pop(0)`	`[5]`	2
`target.append(moved + 10)`	`[5, 12]`	2

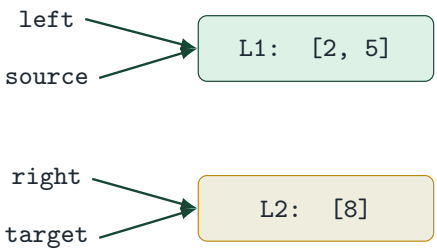
Output

```
[5, 12]
2
```

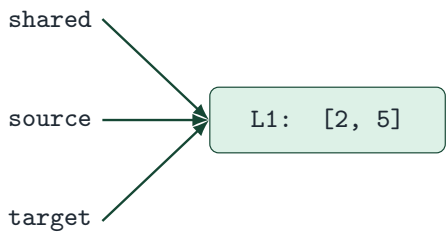
The append acts on the list *after* the removal. Parameter names do not guarantee distinct objects.

Compare the Two Call Diagrams

transfer(left, right): entry



transfer(shared, shared): entry



In class

At function entry, which arrows would you draw for `transfer(shared, shared.copy())`? Which list would `shared` observe afterward?

Quick Check: A Copy as the Second Argument

Start again with `shared = [2, 5]`.

- `shared.copy()` is evaluated before entering `transfer`.
- `source` refers to L1, the original list.
- `target` refers to L2, the new list containing `[2, 5]`.
- The removal changes L1 to `[5]`.
- The append changes L2 to `[2, 5, 12]`.

Caller-visible results

`shared == [5]` and the returned integer is 2.
The caller has not saved a reference to the copied list.

Part 3: Timing and Returned Values

Trace 4: The Copy Happens Too Late

Promise: Return a preview; leave `scores` unchanged.

Code

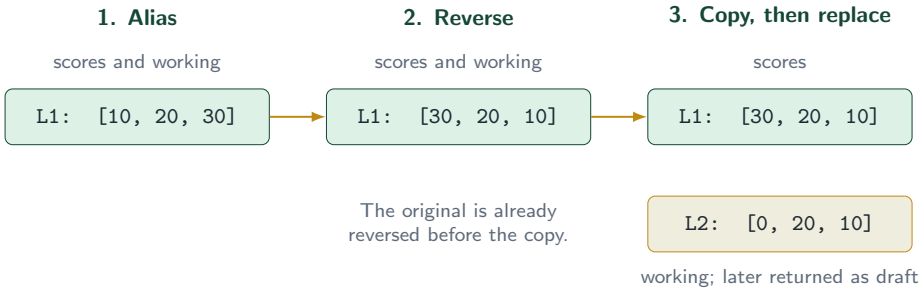
```
def preview(scores: list[int]) -> list[int]:
    working = scores
    working.reverse()
    working = working.copy()
    working[0] = 0
    return working

scores = [10, 20, 30]
draft = preview(scores)
```

In class

Predict both final lists. Identify the first line that breaks the promise. Which statement should move?

Trace 4: A Copy Is a Snapshot of the Current State



Actual result

```
scores == [30, 20, 10]    draft == [0, 20, 10]
```

The copy isolates **later** mutations. It cannot undo a mutation that already reached the original list.

Repair the Promise: Copy Before the First Mutation

Repaired function

```
def preview(scores: list[int]) -> list[int]:
    working = scores.copy()
    working.reverse()
    working[0] = 0
    return working
```

Fresh input: `[10, 20, 30]`

```
scores:  [10, 20, 30]
draft:   [0, 20, 10]
```

In class

Explain why both assignments to `working` in the original are replaced by one assignment here. Which object does `reverse()` change now?

Trace 5: Use the Value Returned by a Mutation

Code

```
queue = [4, 7, 9]
watch = queue
queue.append(queue.pop(0) + len(queue))
print(queue)
print(watch)
```

In class

Trace the expression in order: `pop(0)`, then `len(queue)`, then addition, then `append`. What is the length when it is read?

A single statement can contain more than one mutation. Record intermediate states inside the statement.

Trace 5: Evaluate the Argument Before Appending

Evaluation step	Value produced	Shared list
`queue.pop(0)`	`4`	`[7, 9]`
`len(queue)`	`2`	`[7, 9]`
`4 + 2`	`6`	`[7, 9]`
`queue.append(6)`	`None`	`[7, 9, 6]`

Both outputs

```
[7, 9, 6]
[7, 9, 6]
```

`watch` sees both mutations because its arrow still points to the same list.

Trace 6: Assigning a Method's Result

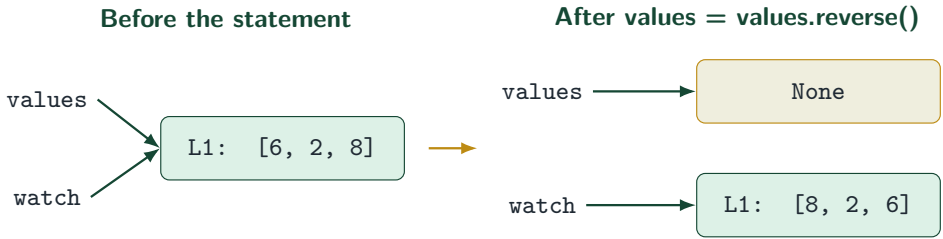
Code

```
values = [6, 2, 8]
watch = values
values = values.reverse()
print(values)
print(watch)
```

In class

Record two events for the third line: what `reverse()` does to the list, and what its returned value makes `values` refer to.

Trace 6: Mutation Happens Before the Assignment



Output

```
None
[8, 2, 6]
```

To keep `values` bound to the reversed list, use `values.reverse()` as a statement. To create a separate reversed list, use `values = values[::-1]`.

Part 4: An Integrated Trace with Two Calls

Challenge Helper: A Mutation and a New List

Function used on the next slides

```
def prepare(source: list[int], extra: int) -> list[int]:
    removed = source.pop(0)
    result = source.copy()
    result.append(removed + extra)
    return result
```

In class

Before seeing the caller, identify which line changes `source`. Does the returned list contain the removed element, the remaining elements, or both in some form?

Assume the supplied list is nonempty. The returned list contains the remaining elements followed by `removed + extra`.

Challenge Caller: Predict Before Seeing the Trace

Code

```
base = [3, 6, 9]
watch = base
first = prepare(base, 1)
second = prepare(first, base[0])
watch.append(second.pop())
first[0] = base[-1]
```

In class

Work in pairs. Predict `base`, `watch`, `first`, and `second`.
Label each new list. Record the integer passed as `extra` in each call.

Also predict `base is watch` and `first is second`.

Challenge: First Call

At `first = prepare(base, 1)`, `source` refers to L1 and `extra` is 1.

Checkpoint	L1	L2	`removed`
entry	`[3, 6, 9]`	—	—
after `pop(0)`	`[6, 9]`	—	3
after `copy()`	`[6, 9]`	`[6, 9]`	3
after `append`	`[6, 9]`	`[6, 9, 4]`	3

Bindings after return

`base, watch -> L1` `first -> L2`

Challenge: Second Call Uses the Current State

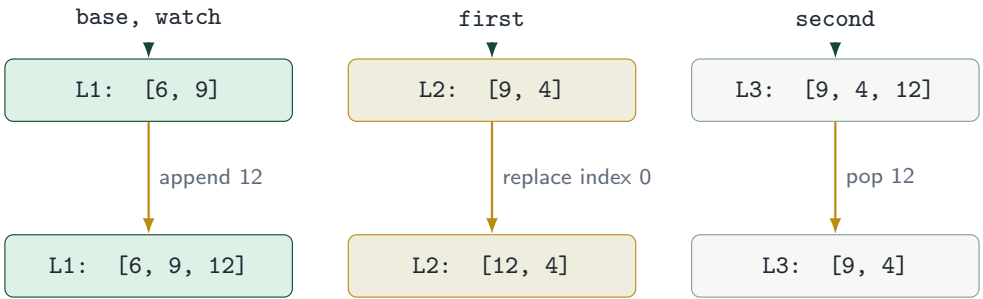
At `second = prepare(first, base[0])`, `base[0]` is now 6. The parameter `source` refers to L2.

Checkpoint	L2	L3	`removed`
entry	`[6, 9, 4]`	—	—
after `pop(0)`	`[9, 4]`	—	6
after `copy()`	`[9, 4]`	`[9, 4]`	6
after `append`	`[9, 4]`	`[9, 4, 12]`	6

Bindings after return

`base, watch -> L1` `first -> L2` `second -> L3`

Challenge: Finish the Last Two Statements



The name-to-list bindings stay the same during the final two statements.

- `watch.append(second.pop())`: remove 12 from L3, then append it to L1.
- `first[0] = base[-1]`: read 12 from L1, then replace index 0 of L2.

Challenge: Final Values and Identity

Name	List	Final contents
`base`	L1	`[6, 9, 12]`
`watch`	L1	`[6, 9, 12]`
`first`	L2	`[12, 4]`
`second`	L3	`[9, 4]`

`base is watch`: `True`. `first is second`: `False`.

In class

Explain why the second call changes `first`, even though its returned value is assigned to `second`.

Change the Contract: Preserve the Argument

In class

Rewrite `prepare` so it returns the remaining elements followed by `removed + extra`, but leaves the argument unchanged. Where must the copy happen?

Change the Contract: Preserve the Argument

In class

Rewrite `prepare` so it returns the remaining elements followed by `removed + extra`, but leaves the argument unchanged. Where must the copy happen?

One repair

```
def prepare_copy(source: list[int], extra: int) -> list[int]:
    result = source.copy()
    removed = result.pop(0)
    result.append(removed + extra)
    return result
```

With `source = [3, 6, 9]` and `extra = 1`, the caller keeps `[3, 6, 9]` and receives `[6, 9, 4]`.

Part 5: Independent Practice

Practice A: A Saved Value and a Snapshot

Code

```
items = [5, 10, 15, 20]
watch = items
saved = items[1]
snapshot = items[1:]
watch.pop(0)
items[1] = saved + snapshot[0]
snapshot.reverse()
```

In class

Predict `items`, `watch`, `saved`, and `snapshot`.
Explain which index 1 changes after the removal. Does `saved` change when the lists change?

Practice A: Reveal

Name	Final value	Reason
`items, watch`	`[10, 20, 20]`	same list; index 1 receives `10 + 10`
`saved`	`10`	integer read before the removal
`snapshot`	`[20, 15, 10]`	separate sliced list, then reversed

Key idea

Saving an element reads its current value. Saving a slice creates a new list. Saving the list name creates an alias.

Practice B: Rebinding Before the Next Mutation

Code

```
def revise(a: list[int], b: list[int]) -> list[int]:
    a.append(b.pop())
    b = b + [0]
    b[0] = 99
    return b

data = [1, 2, 3]
result = revise(data, data)
```

In class

Predict `data`, `result`, and `data is result`. Record the state after `b.pop()` and again after `a.append(...)`.

Practice B: Reveal

1. On entry, `a`, `b`, and `data` refer to L1 = `[1, 2, 3]`.
2. `b.pop()` removes and returns 3; L1 becomes `[1, 2]`.
3. `a.append(3)` puts it back; L1 becomes `[1, 2, 3]`.
4. `b = b + [0]` creates L2 = `[1, 2, 3, 0]` and rebinds `b`.
5. `b[0] = 99` changes L2, which is returned.

Final result

`data == [1, 2, 3]` `result == [99, 2, 3, 0]`
`data is result` is `False`.

L1 was mutated twice even though its final contents match its initial contents.

Exit Ticket: Explain Every Arrow

Code

```
original = [4, 8]
alias = original
copy = original.copy()
answer = alias.append(copy.pop())
alias = copy
alias[0] = 0
```

In class

Give the final values of all four names. Which names share a list at the end? Identify one mutation and one rebinding, explaining their different effects.

Exit Ticket: Reveal

Name	Refers to	Final value
`original`	L1	`[4, 8, 8]`
`alias, copy`	L2	`[0]`
`answer`	`None`	`None`

- `copy.pop()` removes 8 from L2; `append` adds it to L1 and returns `None`.
- `alias = copy` moves one arrow from L1 to L2.
- `alias[0] = 0` changes L2, visible through `copy` too.

What a Complete Mutation Trace Shows

- The current list object for each active name and parameter.
- Intermediate contents after *each* mutation, including inside expressions.
- The returned value separately from any change to an argument.
- A new box whenever slicing, copying, or concatenation creates a list.
- A moved arrow whenever a name is rebound.
- The exact first mutation that violates a function's promise.

Key idea

Follow the current arrows, execute one step at a time, and place a protective copy before the first mutation that needs isolation.